



Not Only SQL

- IIC2173 - Arquitectura de sistemas de software
- Departamento de Ciencia de la Computación
- Escuela de Ingeniería – PUC
- Nicolás Acosta – Gerardo Crot



Que se ha visto hasta hoy

- Conceptos de Arquitectura de Sistemas
- Características arquitectónicas
- E&P
- Integración
- Performance
- Escalabilidad

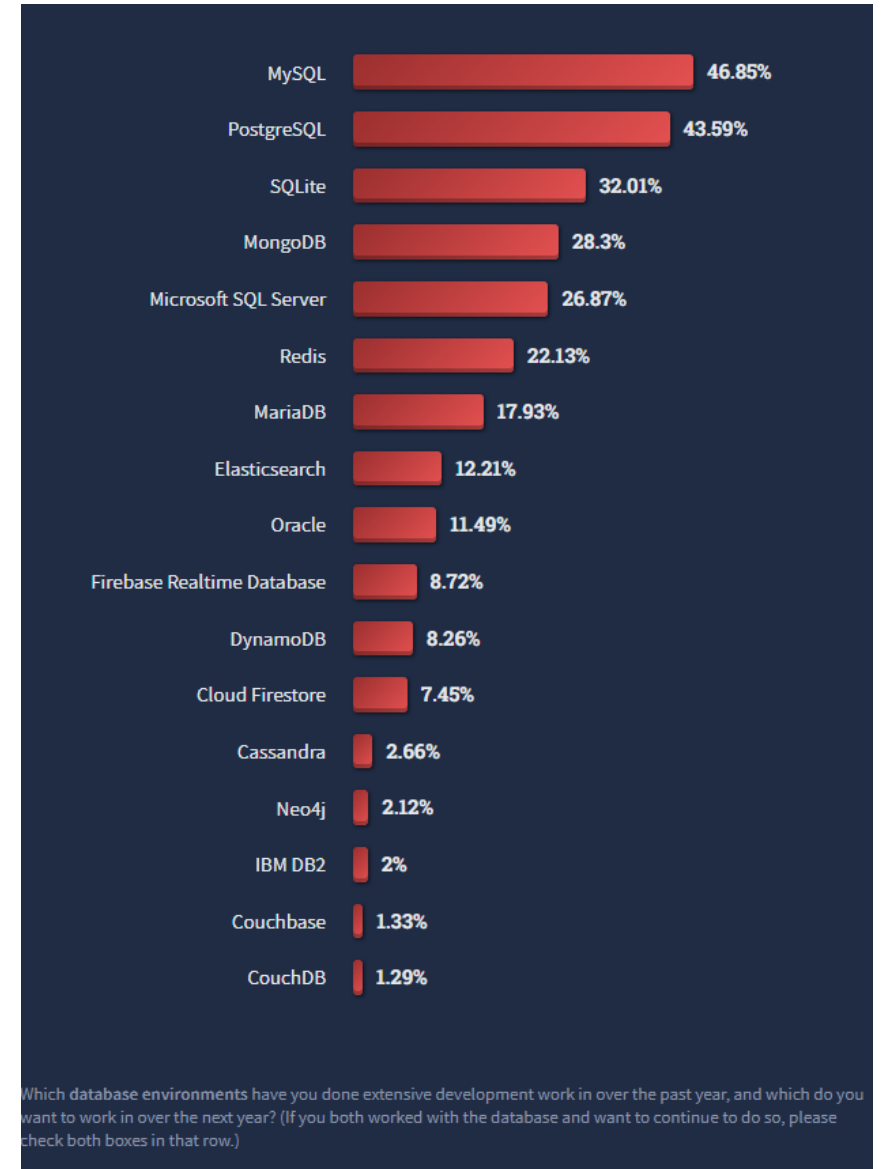


Que queda por ver

- Hoy
 - No SQL
- ¿Qué queda?
 - Seguridad
 - Deploys y Monitoreo

El dominio de SQL

- SQL predominó fuertemente el mercado
 - Y su legado continúa
- Sus características permiten solucionar varios temas
 - Estandarización
 - **Modelo relacional**, “versatilidad”
 - **ACID**, soporte transaccional
 - Datos en *Schema*
 - Empresas aman las tablas
 - Soporte *queries* complejas, OLAP
 - Madurez y soporte
 - **Interoperable** (ORM, LINQ, etc)



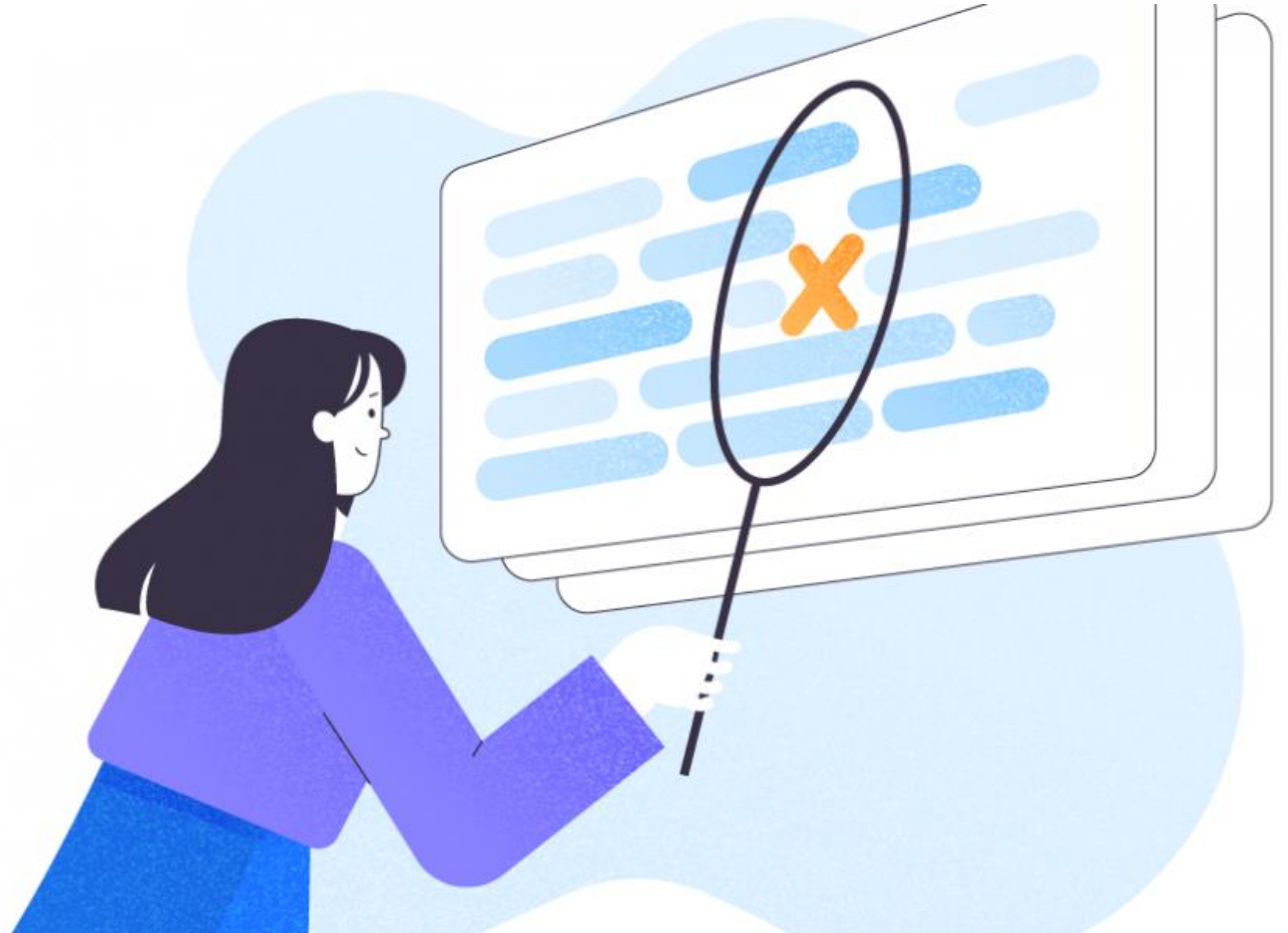
Características de SQL

- Declarativo
 - “Quiero que el sistema me de este resultado”
 - Oculta complejidad al programador
- Basado en lenguajes
 - DDL – *Data Definition Language*
 - CREATE, DROP, Alter
 - DQL – *Data Query Language*
 - SELECT (joins!)
 - DML – *Data Manipulation Language*
 - INSERT, UPDATE, DELETE
 - DCL – *Data Control Language*
 - GRANT (CBAC)
 - TCL – *Transaction Control Language*
 - BEGIN, COMMIT, ROLLBACK

¿Qué pasa con SQL?

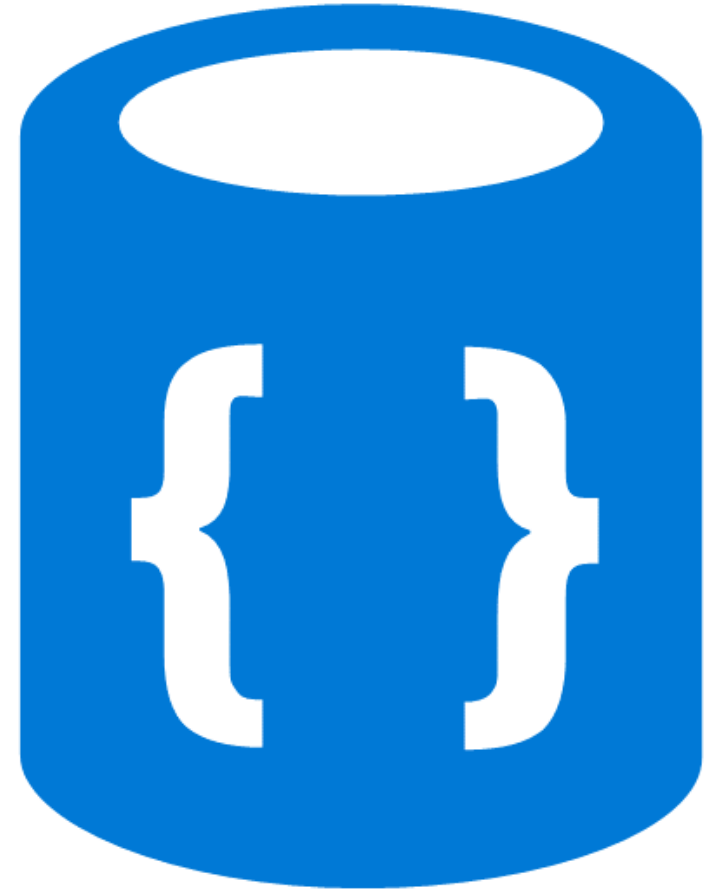
- *Schema* rígido dificulta evolución
- "Tunning" solo para expertos
- *Objet-relationship mismatch*
 - Relaciones muy estructuradas
 - Puede surgir "Impedancia" en la traslación de data
- Complejidad y eficiencia impredecible
- Dificultades en escalabilidad
 - Sharding/Partitioning tiene un límite
 - Flujo común:
Crecimiento -> Distribución física ->
Replicación de datos (desnormalización)

¡Ya no es relacional!



Not Only SQL

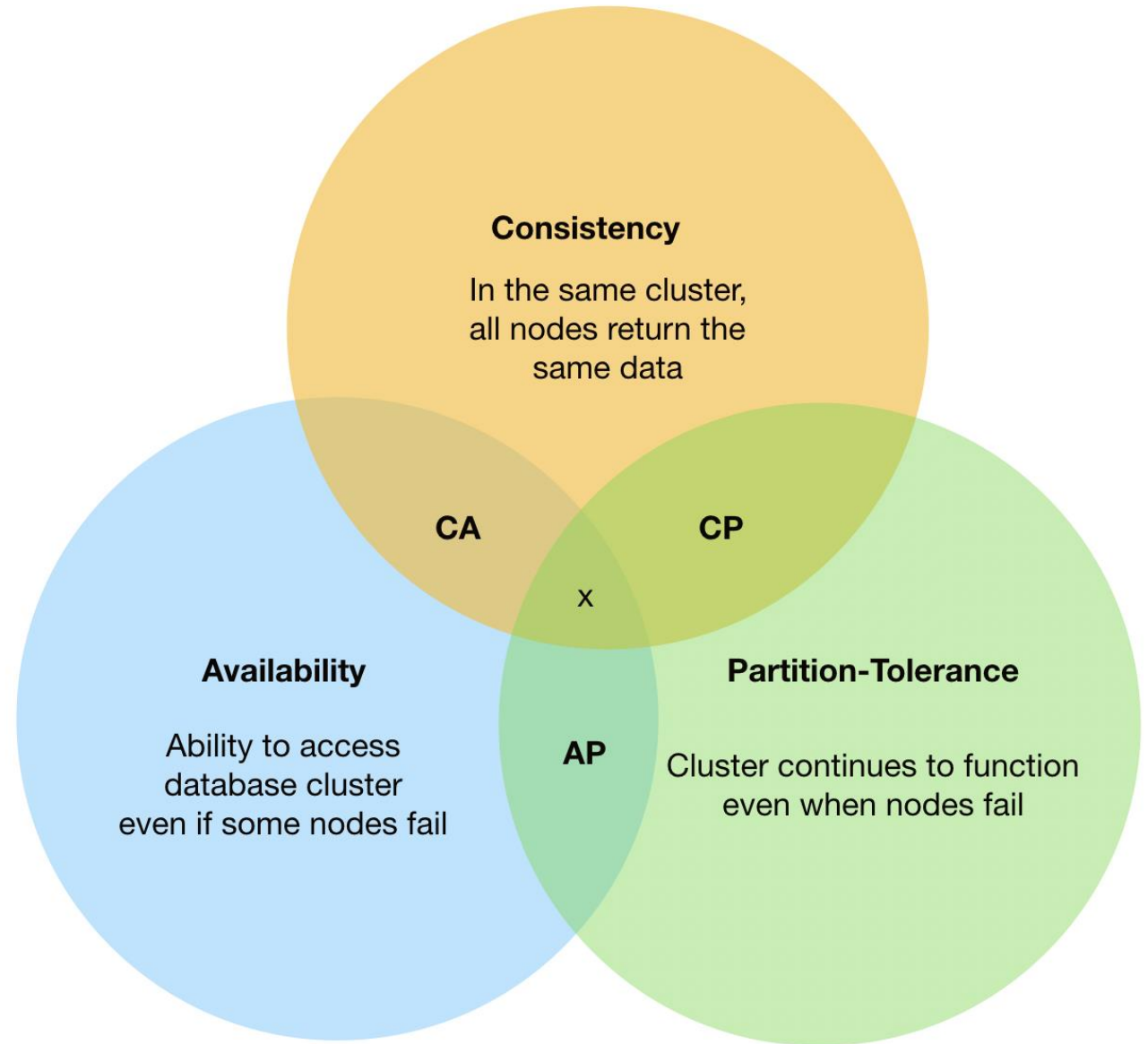
- Metas:
 - Simplificar la manipulación de datos con tal de poder predecir el performance de las queries
 - La complejidad de las queries las maneja el developer
 - Abordar otras estructuras de datos
 - *Schema-less* (o schema implícito)
- Se rige por las propiedades BASE
 - Basically Available
 - Soft state
 - Eventually consistent



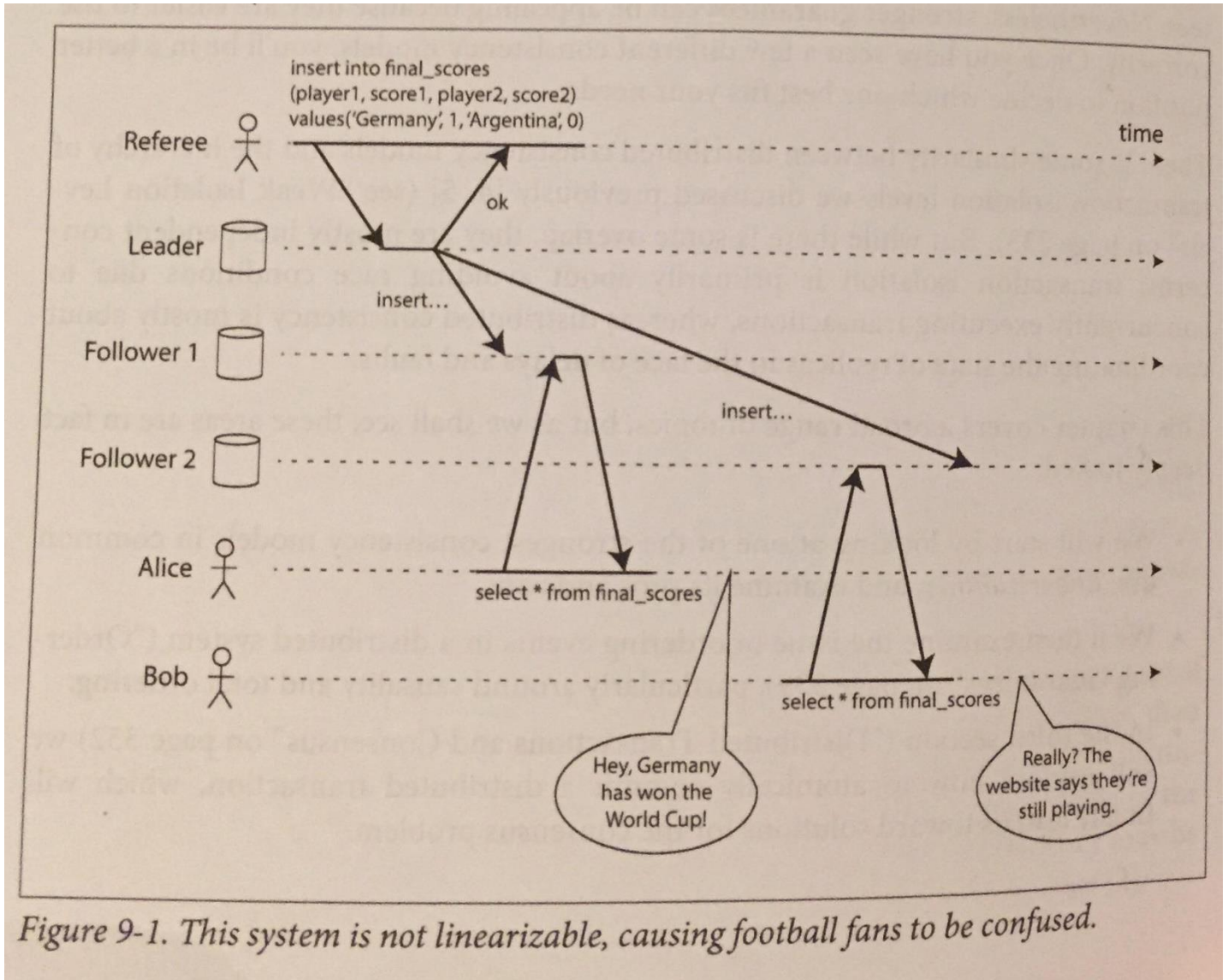
¿Por qué surge NoSQL?

- Crecimiento masivo de datos
 - Exabytes x año (2010) (1000 PB)
 - Zettabytes x año (2020) (1000 EB) (WEF, Statista)
- Conectividad
 - Web 1.0: Texto, hipertexto, wikis, RSS
 - Web 2.0: Blogs, Contenido generado por usuario, tagging, RDF
 - Web 3.0: Ontologías? Folksonomias? GGG (global Giant Graph)
- Datos semi-estructurados
- Arquitectura
- Limitaciones en el performance de RDBMS

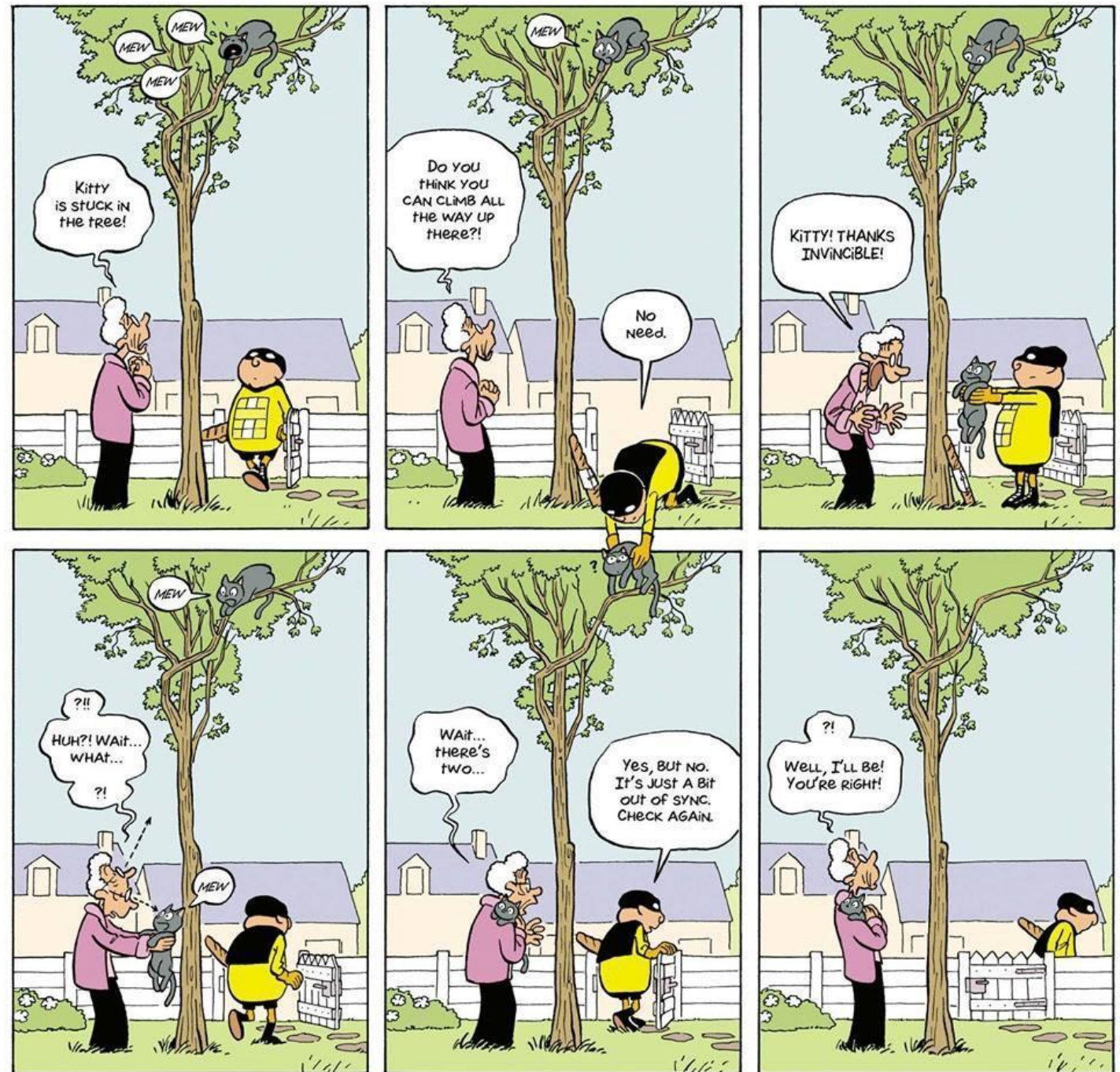
Sobre el teorema CAP



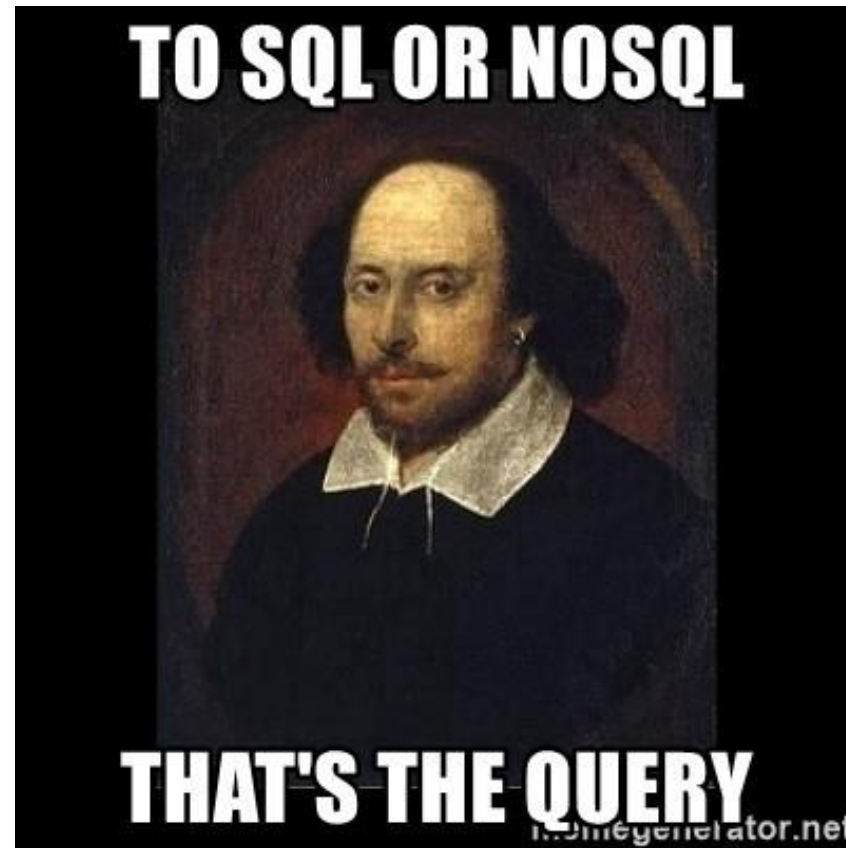
Dejando ir a CAP



Eventualmente consistente (BASE)



Entonces, la gran pregunta: ¿SQL o NoSQL?



Categorías de NoSQL

- Key-Value
- Columnar (Tabular)
- Documentos
- Grafos
- Vectorial
- Multischema

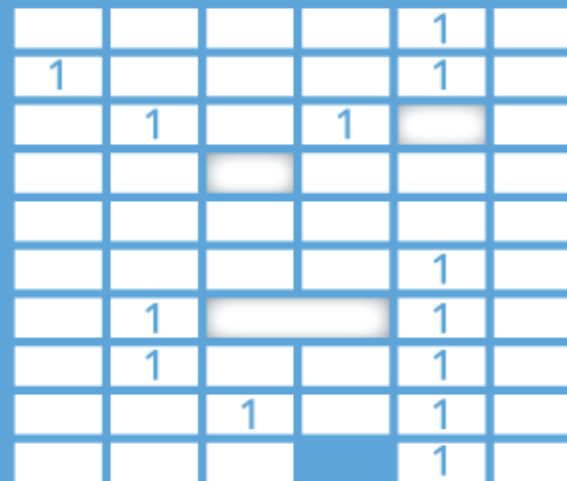
Key-Value



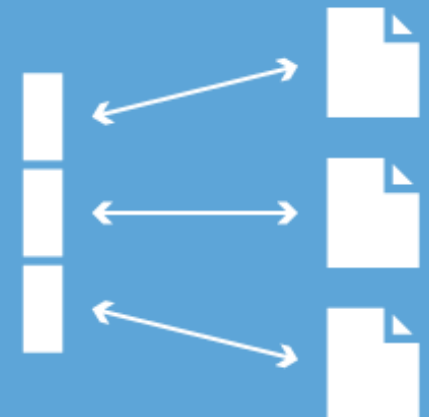
Graph DB



Column Family

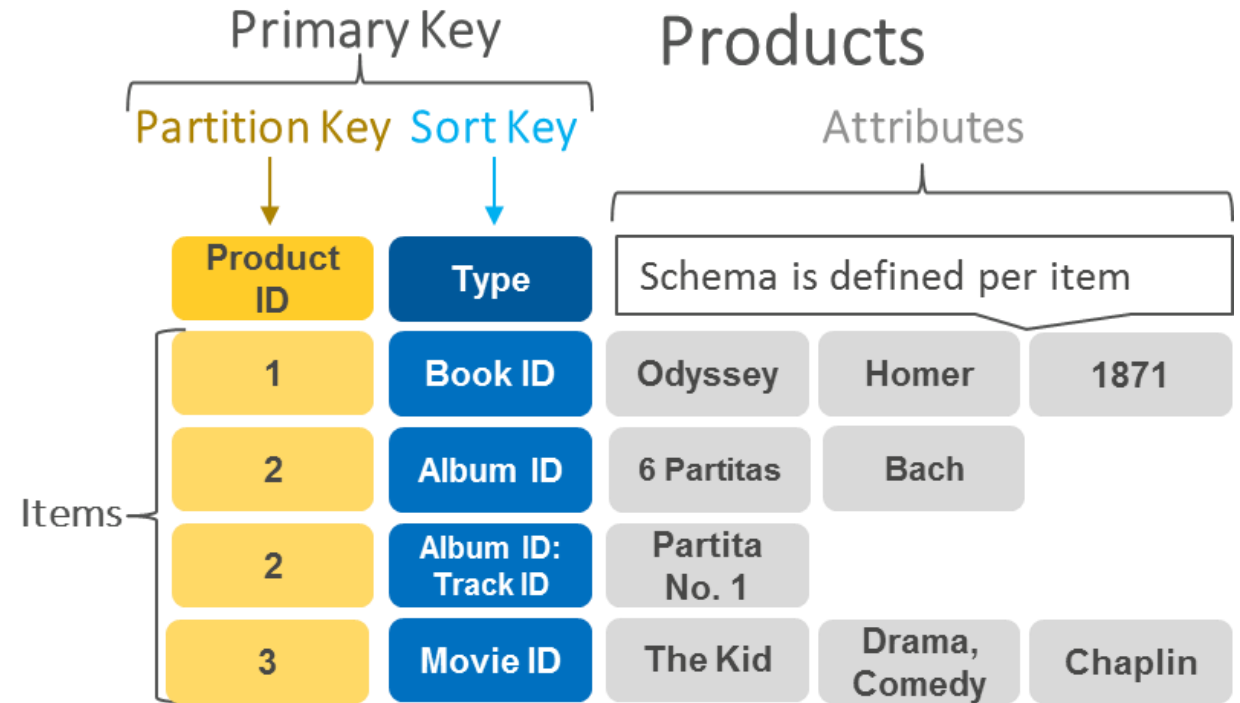


Document

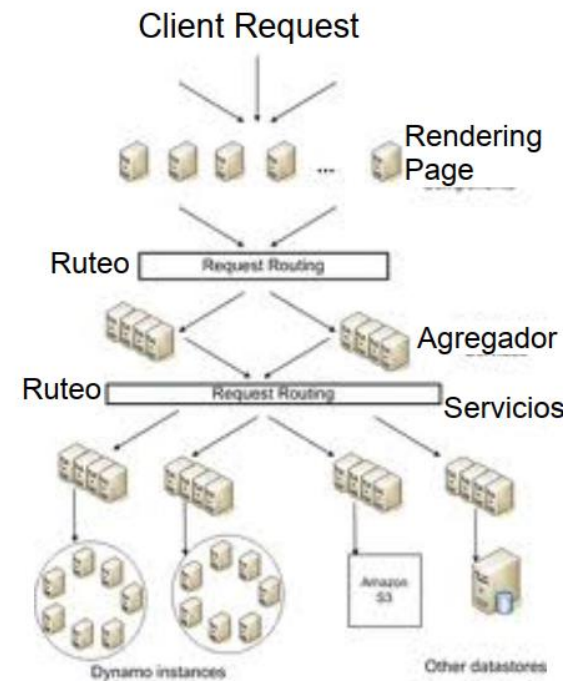
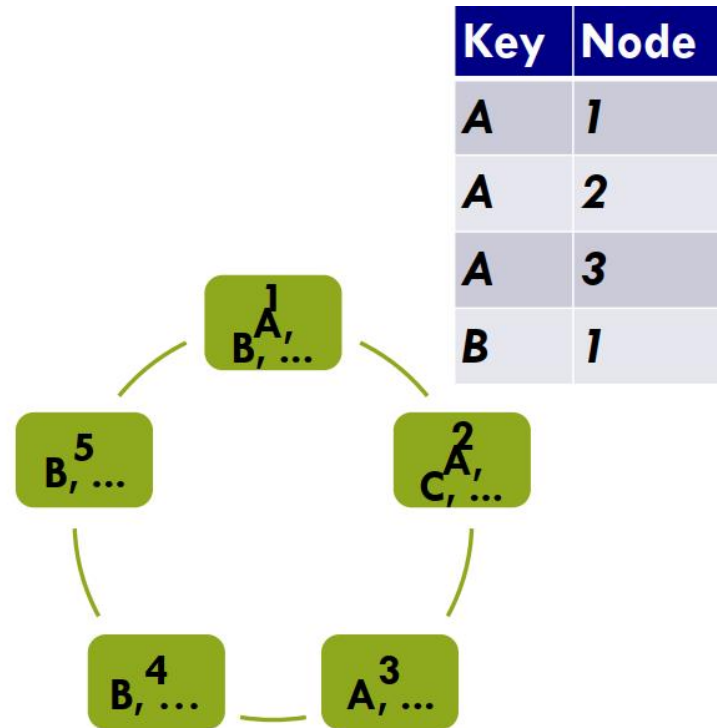


Key-Value

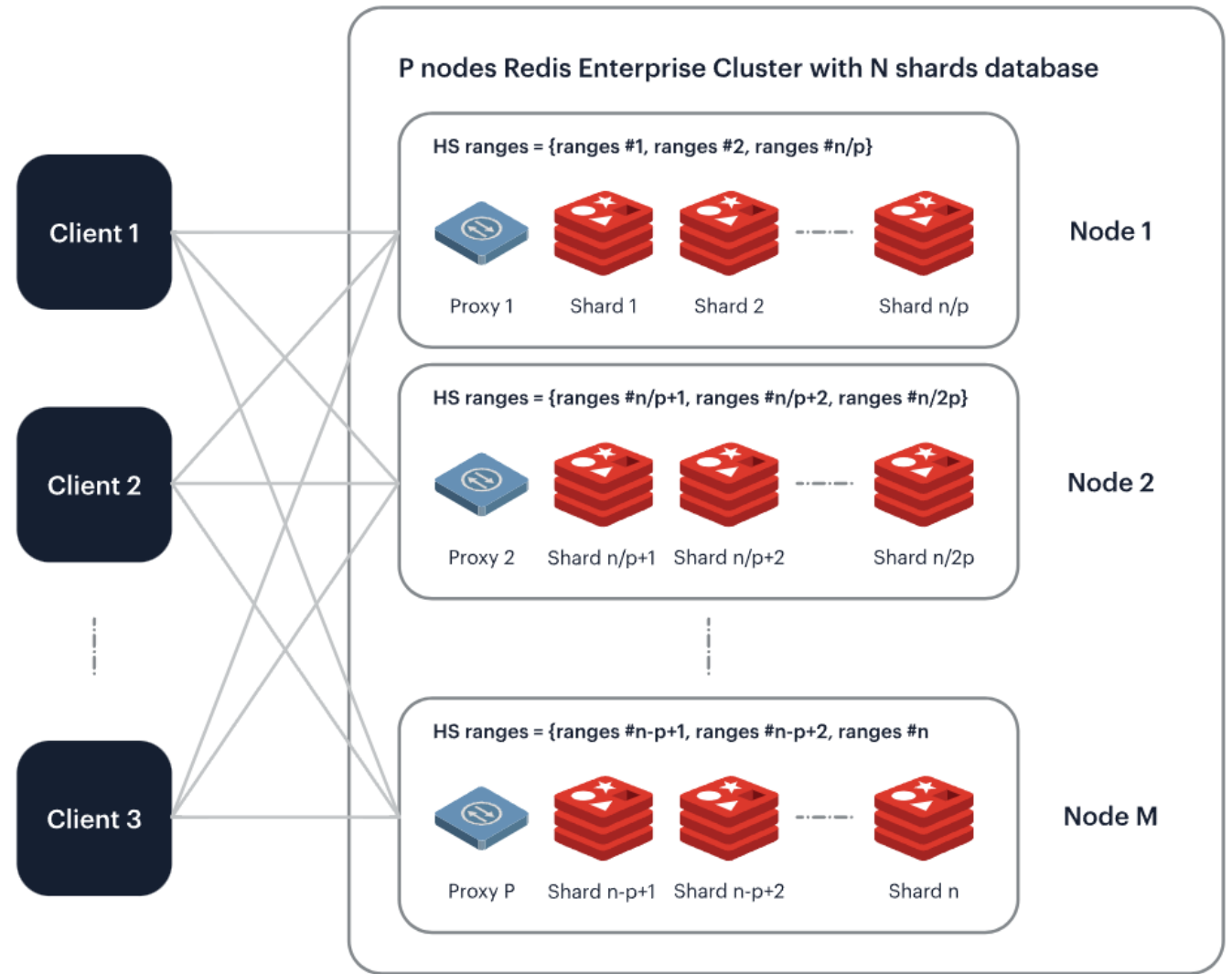
- Modelo de datos
 - Colección de **pares key-value**
- Foco
 - Escalar enormes cantidades de datos
 - Datos simples, sin *schema*
 - Manejar cargas masivas
 - Reducir latencia, elevar performance
- Categorías
 - Distribuidas: Basadas en Dynamo
 - In-memory/persistent: Redis



Dynamo: Anillo de Hash

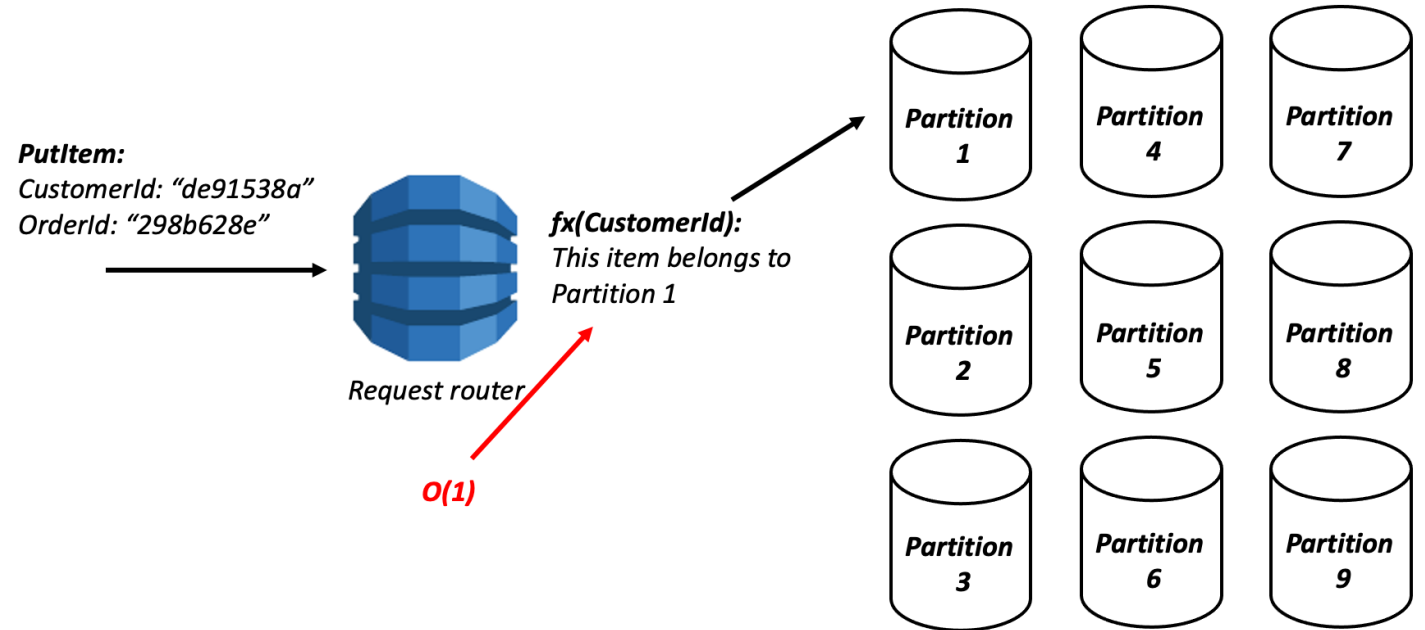


Redis: Master/slave



Pros/cons

- Pros
 - Muy **simple** de manejar
 - Altamente **escalable**
 - Performance elevado (ibien usado!)
- Cons
 - **Mínima estructura** disponible
 - Puede que la data tenga un límite (Dynamo: 400KB)
 - Ausencia de Joins y queries más complejas
 - Consistencia eventual (en algunos)
 - Uso de **RAM** (Redis)



Vendors

- Eventualmente consistente
 - DynamoDB
 - Voldemort
- RAM Cached
 - Memcached
 - Oracle Coherence
 - Velocity
- Disk Cached
 - MemcacheDB
- Dual
 - Redis

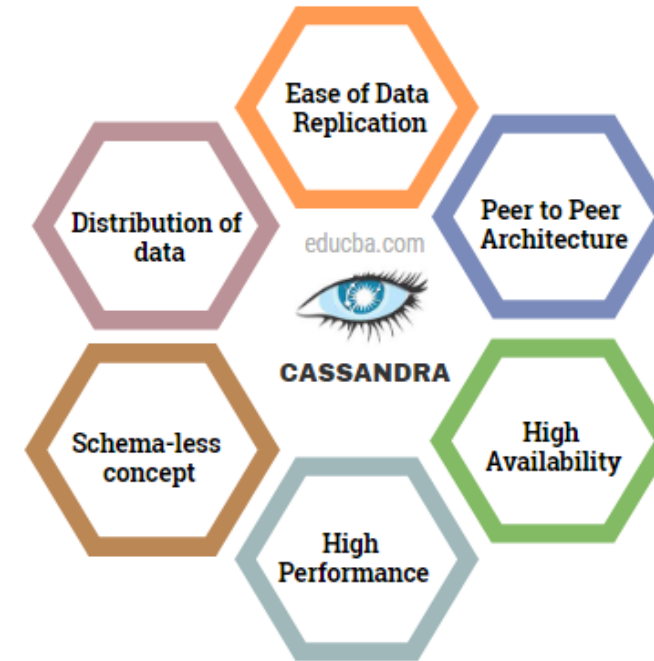


amazon
DynamoDB

Columnar (Tabular)

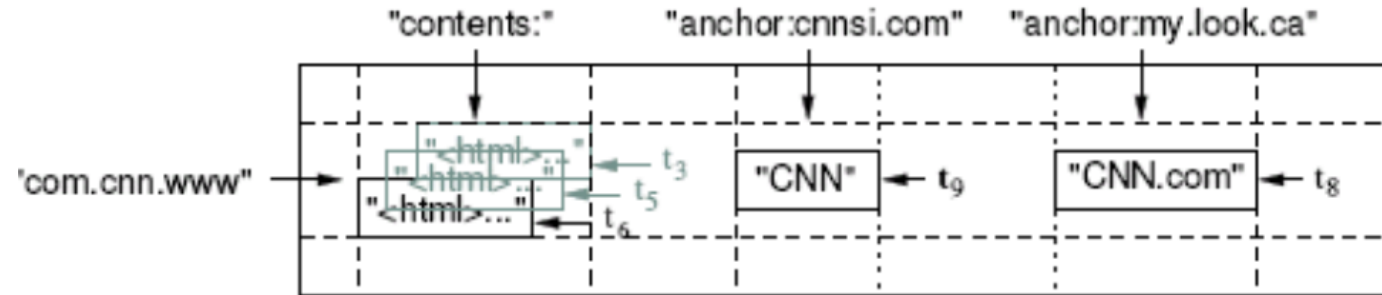
- Tablas similares a RDBMS, pero maneja datos semiestructurados
- Modelo de datos:
 - Columnas → Familias de columnas → ACL
 - Datos tienen Keys de: fila, columna, tiempo, índice
 - Rango de filas (tableta) → tabla → distribución
- Foco en gestión de grandes cantidades de datos
- Ejemplos
 - Google BigTable, Hbase, Hypertable, Cassandra

What is Cassandra

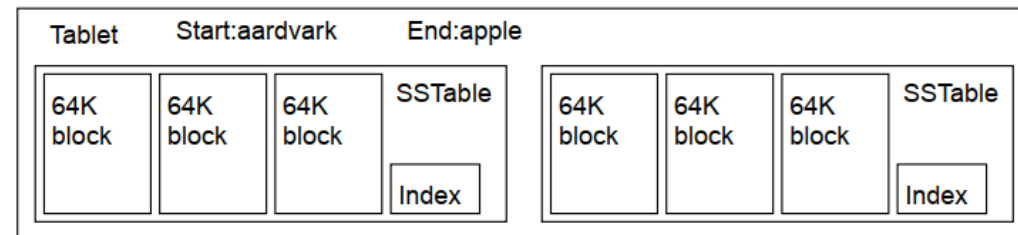


Google BigTable

- Cubo tridimensional: <rowKey, columnKey, Timestamp>

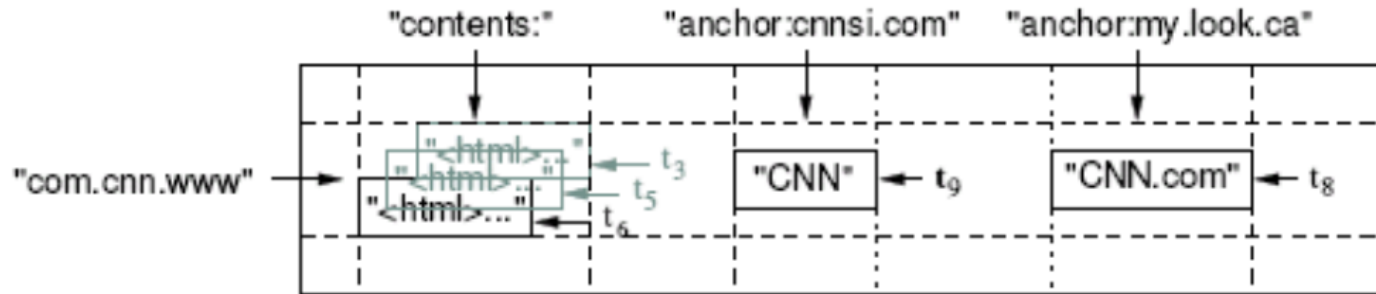


- Des-agregación: Tableta = rango de filas (SSTable)

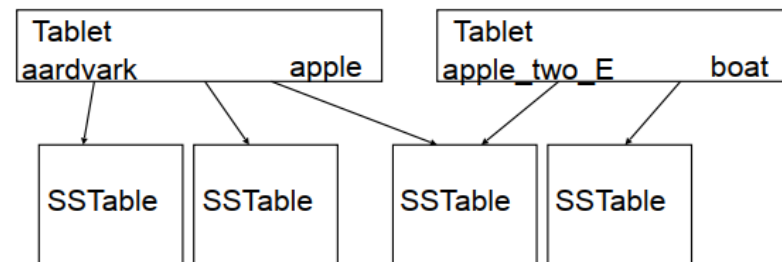


Google BigTable

- Cubo tridimensional: <rowKey, columnKey, Timestamp>

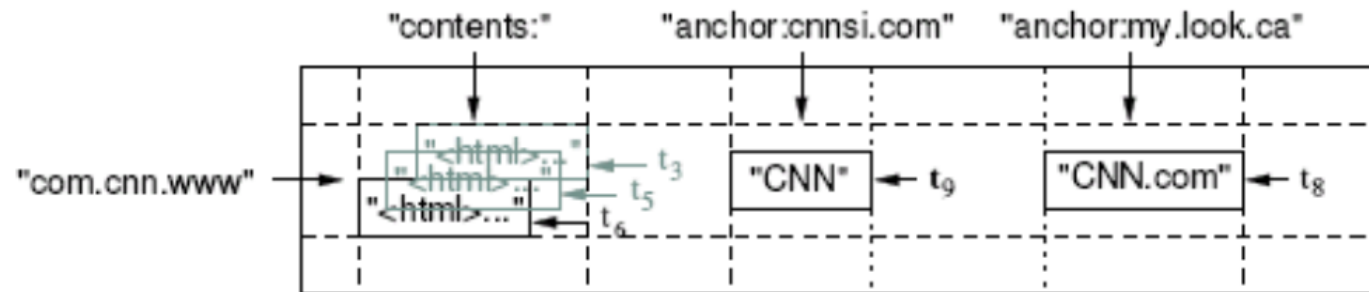


- Des-agregación: Tabla = conjuntos de tabletas

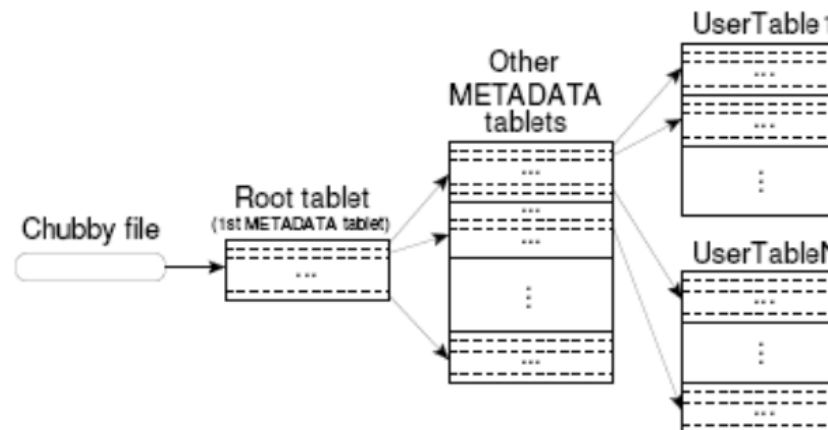


Google BigTable

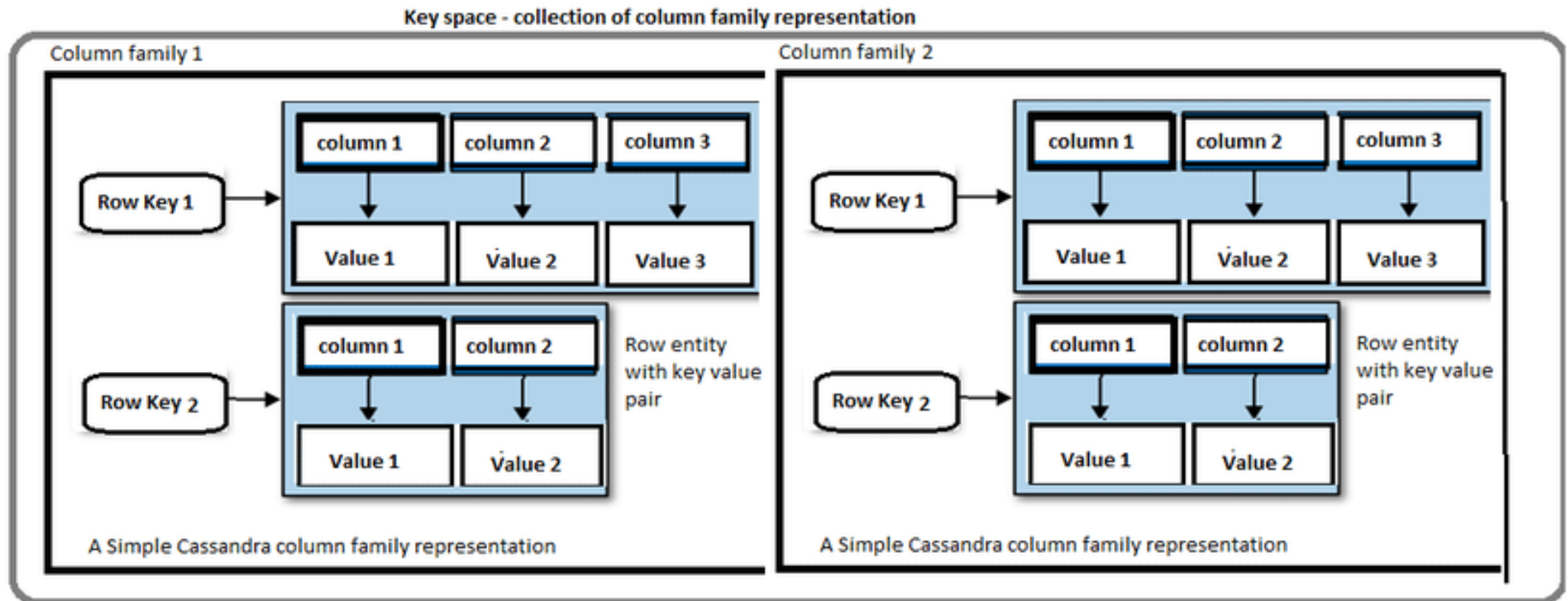
- Cubo tridimensional: <rowKey, columnKey, Timestamp>



- Des-agregación:
Servidores =
conjuntos
de tablas



Apache Cassandra



Vendors

- BigTable
- Apache Hbase+Hadoop
- Cassandra
- Hypertable



Cloud
Bigtable

Documental

- Similar a Key-Value pero la DB sabe cuál es el value
 - Un documento, formato JSON/XML/etc.
- Modelo de datos
 - Colecciones de colecciones de pares llave-valor
 - Así no hay colisiones de keys
 - Índices
 - Bases de datos
- Documentos suelen estar versionados

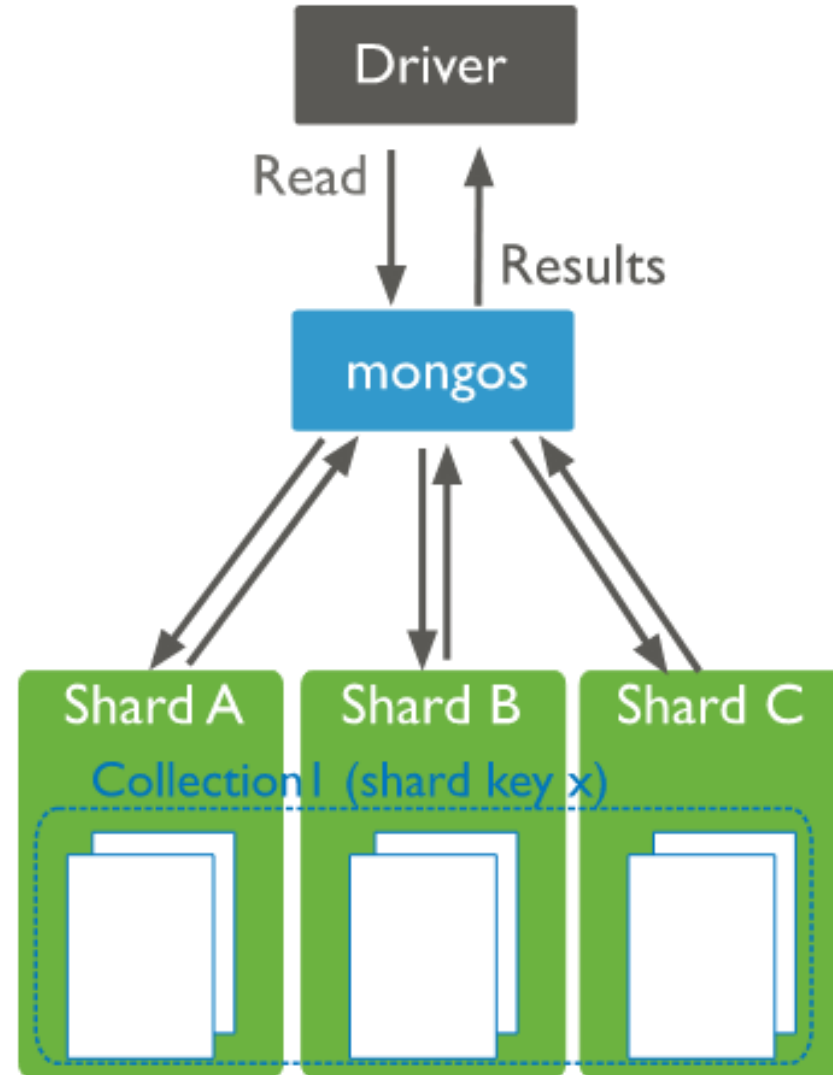
```
{
  _id: <ObjectId>,
  username: "123xyz",
  contact: {
    phone: "123-456-7890",
    email: "xyz@example.com"
  },
  access: {
    level: 5,
    group: "dev"
  }
}
```

Embedded sub-document

Embedded sub-document

MongoDB

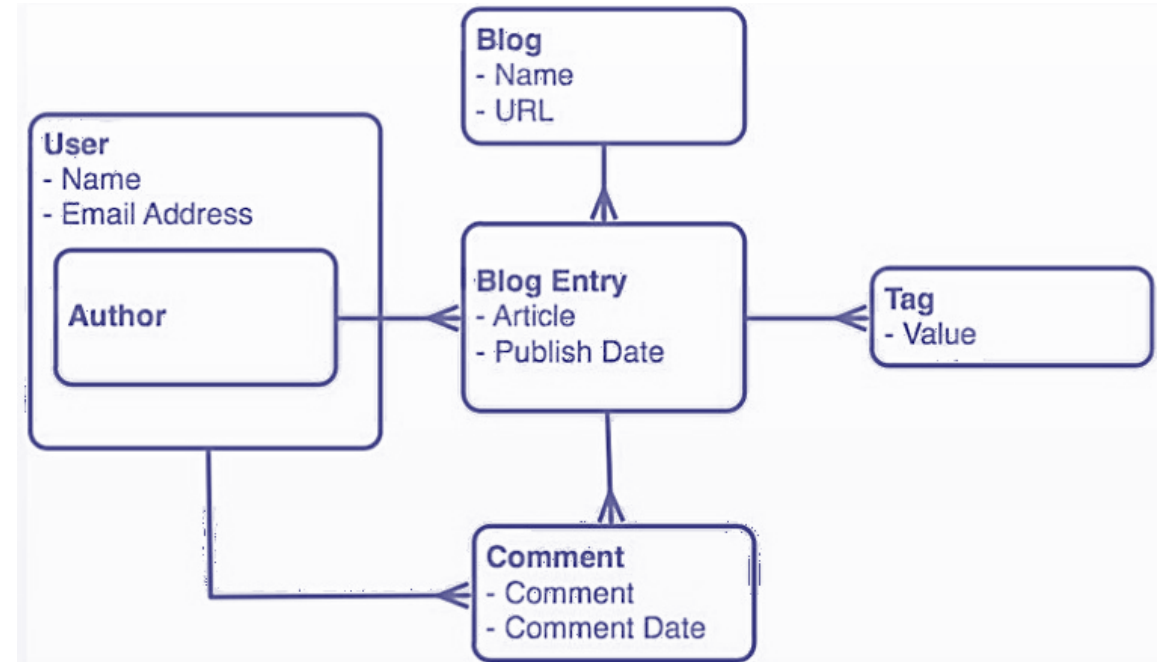
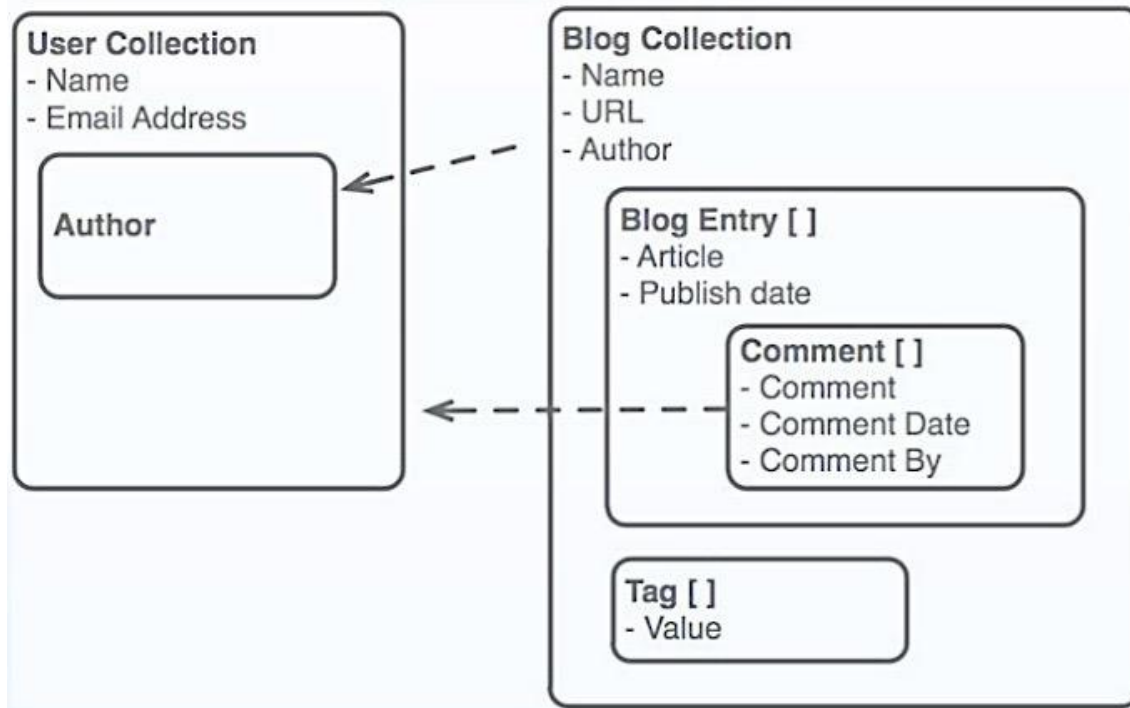
- Formato de almacenamiento:
Documentos JSON
 - Documentos BSON
- Replicación y alta disponibilidad
- Auto-Sharding
 - Escalamiento horizontal (documentos: es como si fuera por filas no por columnas)
- Querying, indexing, updating
 - Update: In-place, fast
 - Índices en cualquier atributo
- Map/Reduce
- Grid File System



Relacional vs Documentos

RDBMS	Mongo
Table, View	Collection
Row	JSON Document
Index	Index
Join	Embedded
Partition	Shard
Partition Key	Shard Key

Relacional vs Documentos



Mongo vs SQL

```
UPDATE users
SET status = 'C'
WHERE age > 25;
```

```
db.users.updateOne(
  { age: { $gt: 25 } },
  { $set: { status: "C" } },
  { multi: true }
)
// see note below table.
```

```
START TRANSACTION;
INSERT INTO orders
(order_id, product, quantity)
VALUES ('1a2b3c', 'T-shirt', '7');
UPDATE stock
SET quantity=quantity-7
WHERE product='T-shirt';
COMMIT;
```

```
session.startTransaction();
db.orders.insert ({
  order_id: '1a2b3c',
  product: 'T-shirt',
  quantity: 7
})
db.stock.updateOne (
  { product: { $eq: 'T-shirt', } },
  { $inc: { quantity: -7 } }
)
session.commitTransaction();
```

SQL

```
CREATE TABLE users (
  user_id VARCHAR(20) NOT NULL,
  age INTEGER NOT NULL,
  status VARCHAR(10));
```

```
INSERT INTO users(user_id, age, status)
VALUES ('bcd001', 45,"A");
```

```
SELECT *
FROM users;
```

MongoDB

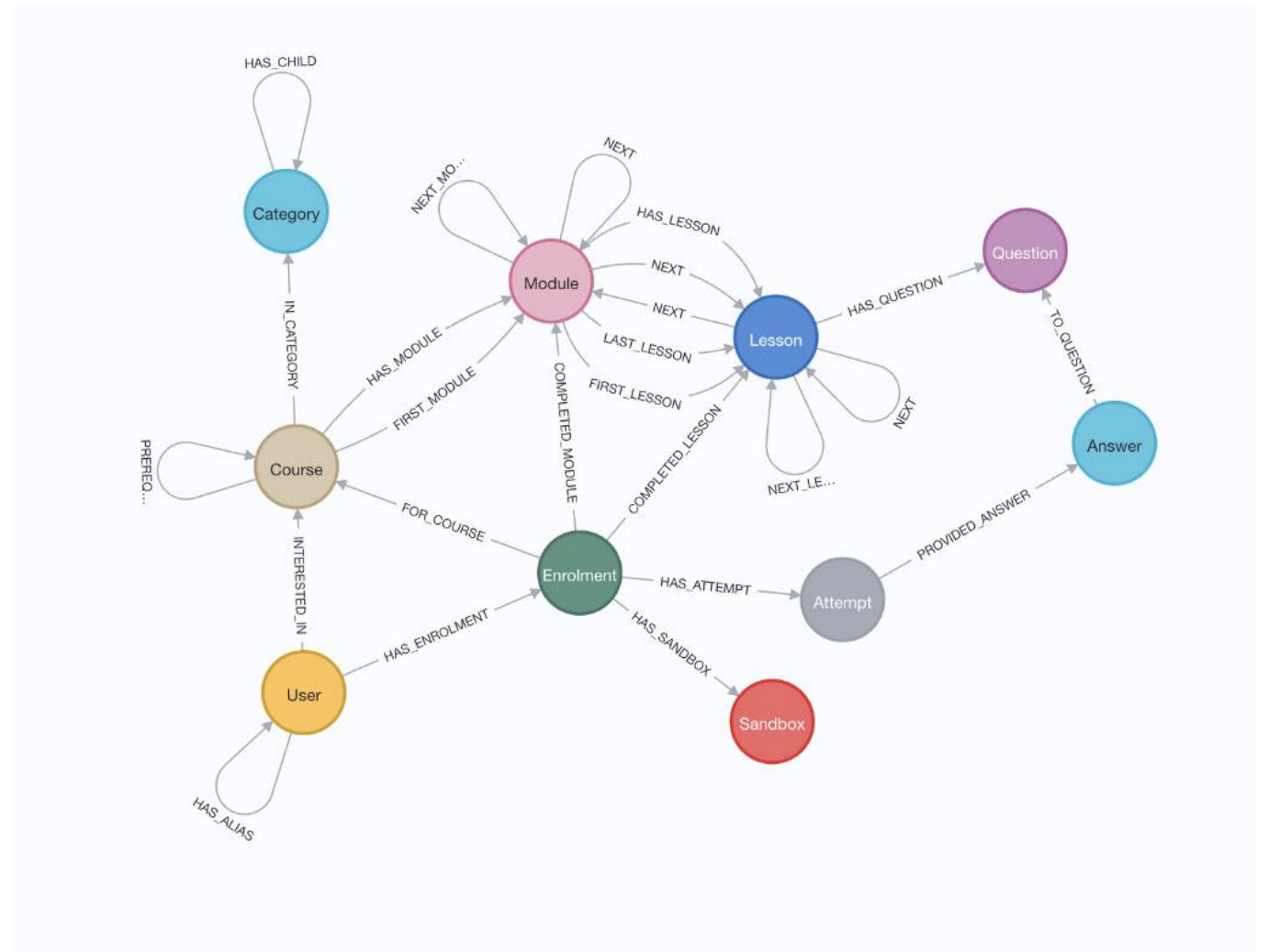
Implicitly created on first `insertOne()` or `insertMany()` operation. The primary key `_id` is automatically added if `_id` field is not specified. However, you can also explicitly create a collection: `db.createCollection("users")`

```
db.users.insertOne({
  user_id: "bcd001",
  age: 45,
  status: "A"
})
// see note below table.
```

```
db.users.find()
```

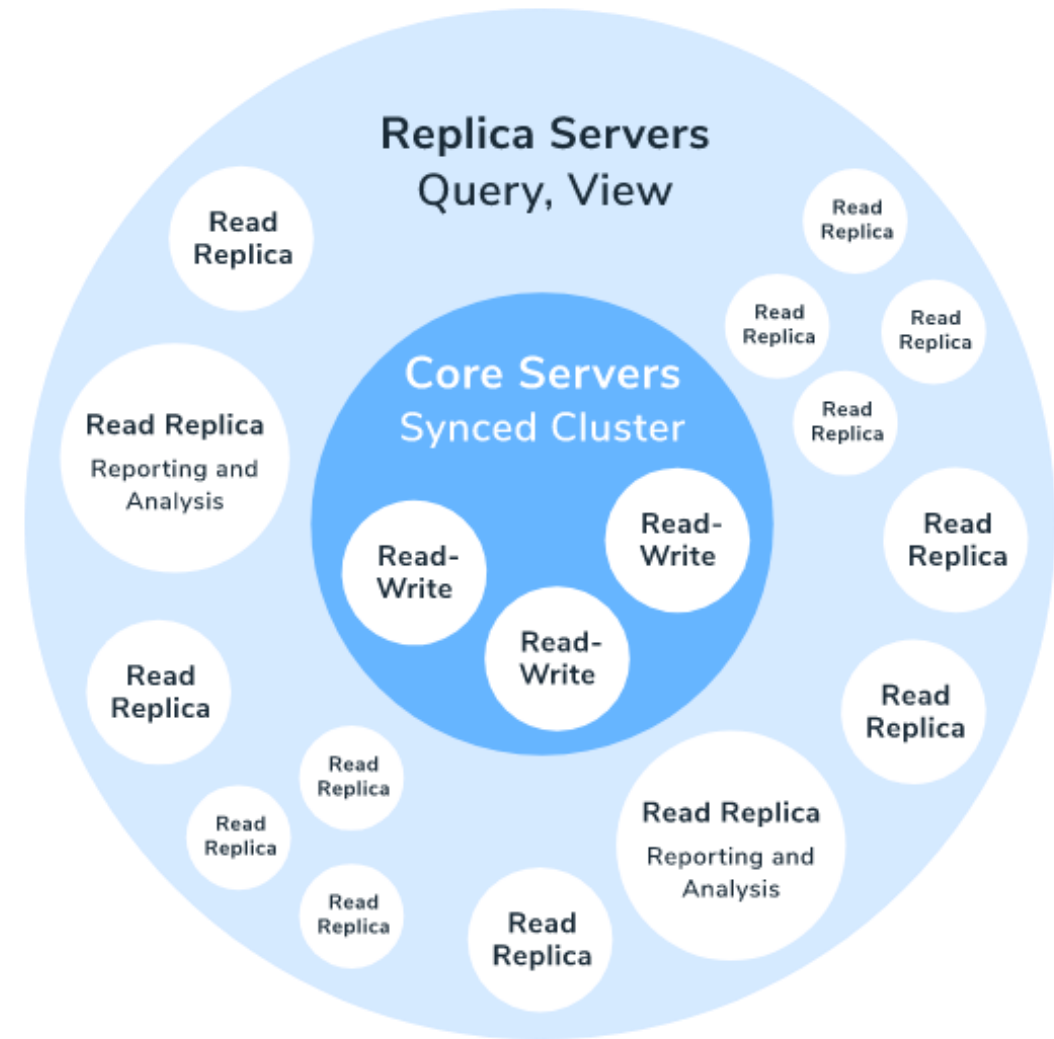

Grafos

- Inspiradas en la teoría de grafos
 - Modelo de datos: Grafo de propiedades
 - Nodo: First-class citizen
 - Relaciones/Arcos entre Nodos
 - Pares llave-valor para ambos
 - Arcos pueden tener etiquetas o tipos
 - Neo4J, AllegroGraph



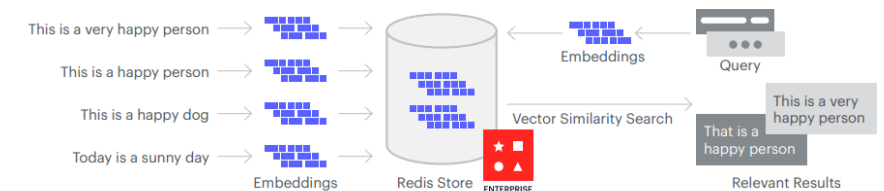
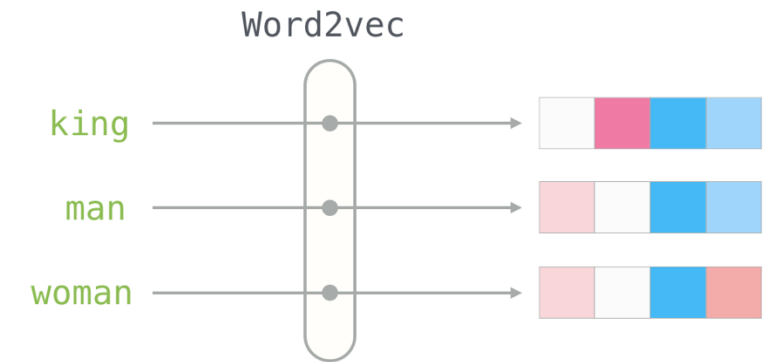
Grafos: Pros/Cons

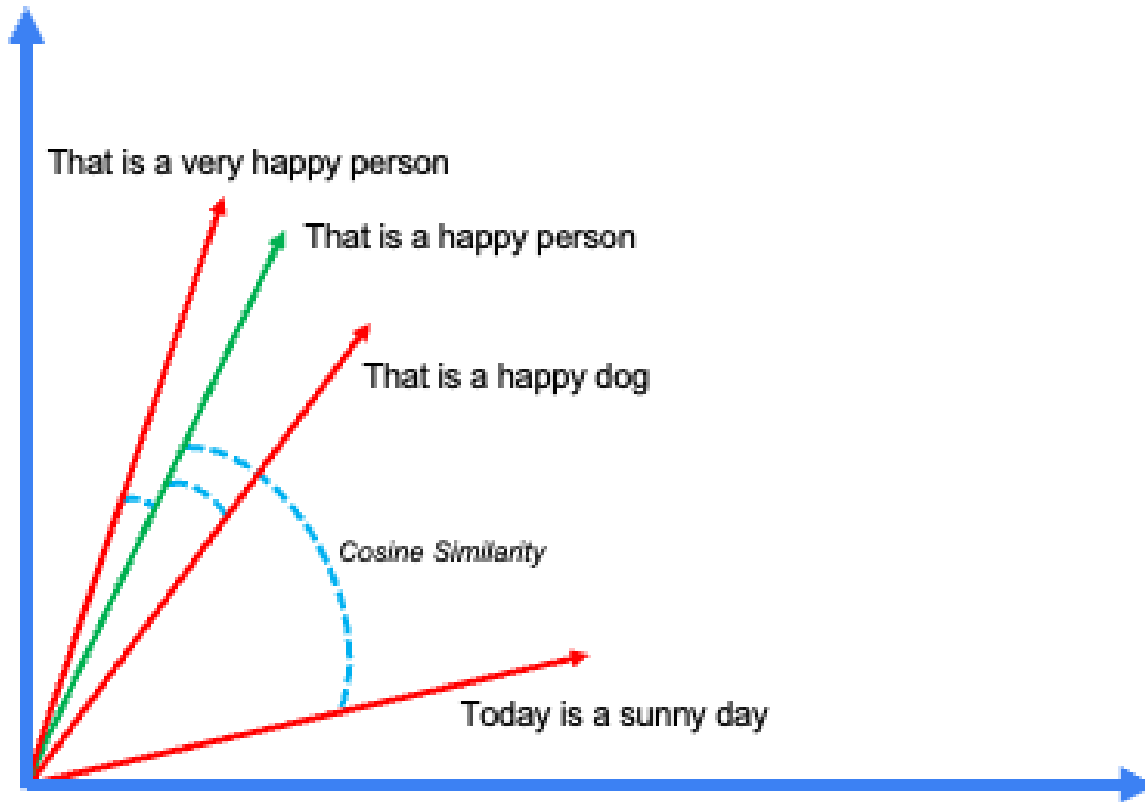
- Pros
 - Posibilidad de representar grafos
 - Mejores en relaciones complejas (sobre SQL y otros)
 - Mejor performance (sobre hacerlo en SQL)
- Cons
 - Más **complejo** de aprender
 - Overhead adicional
 - Puede ser complejo escalar
 - Sharding, manejo de clusters



Vectorial

- Usado en AI, para **búsquedas de similitud**
- Codifica datos en “embeddings”
 - Texto
 - Videos
 - Audios
- Alta demanda para modelos GPT y sistemas recomendadores

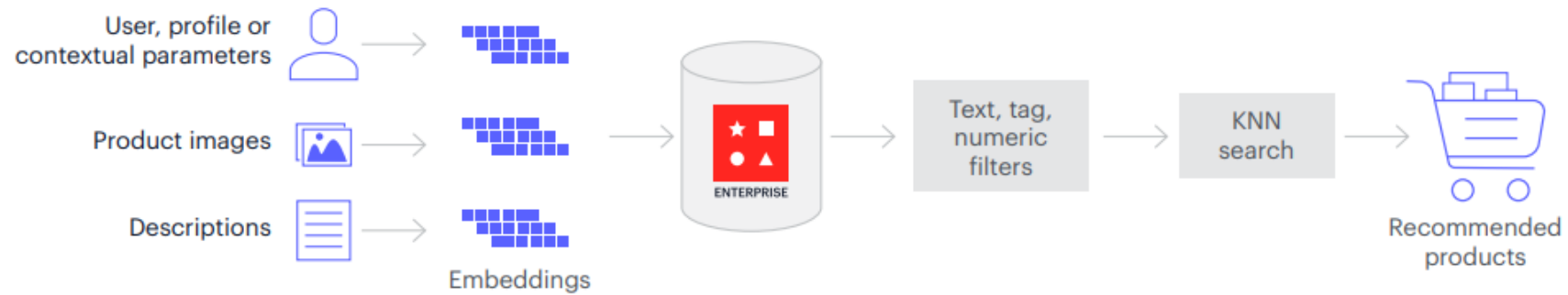




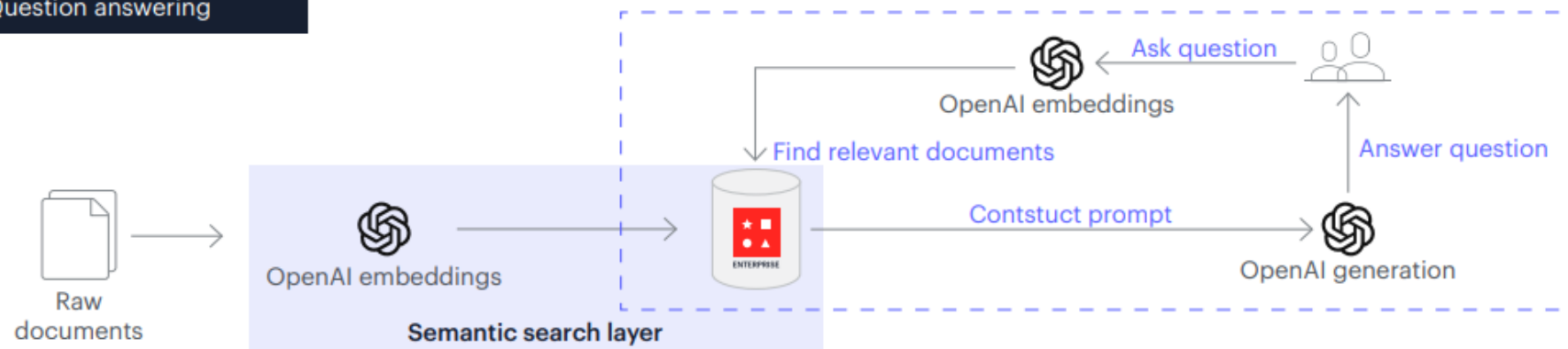
Vectores y embeddings

Casos de uso

Use case: Recommendation systems



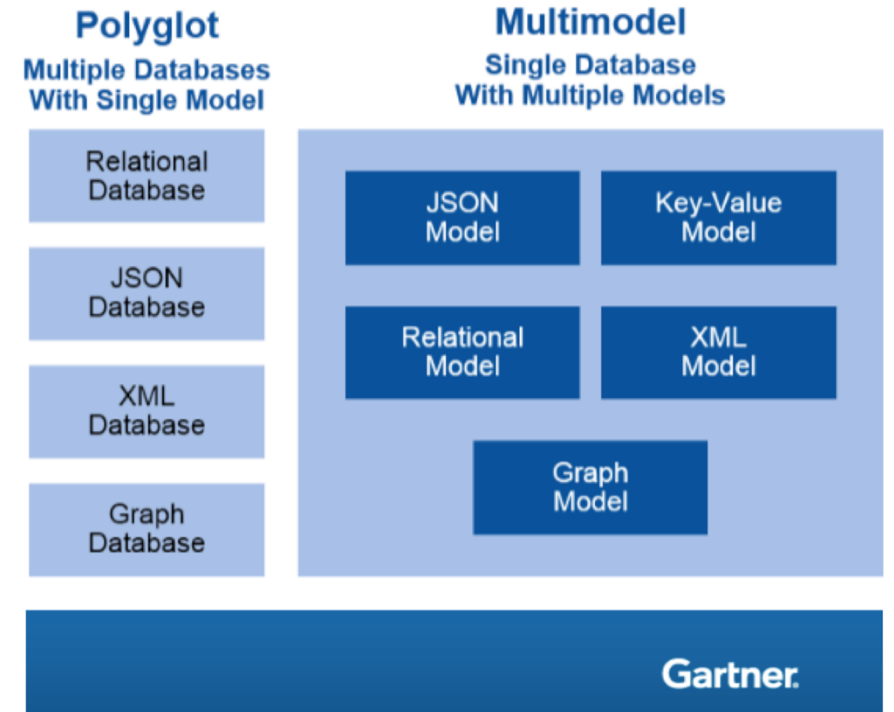
Use case: Question answering



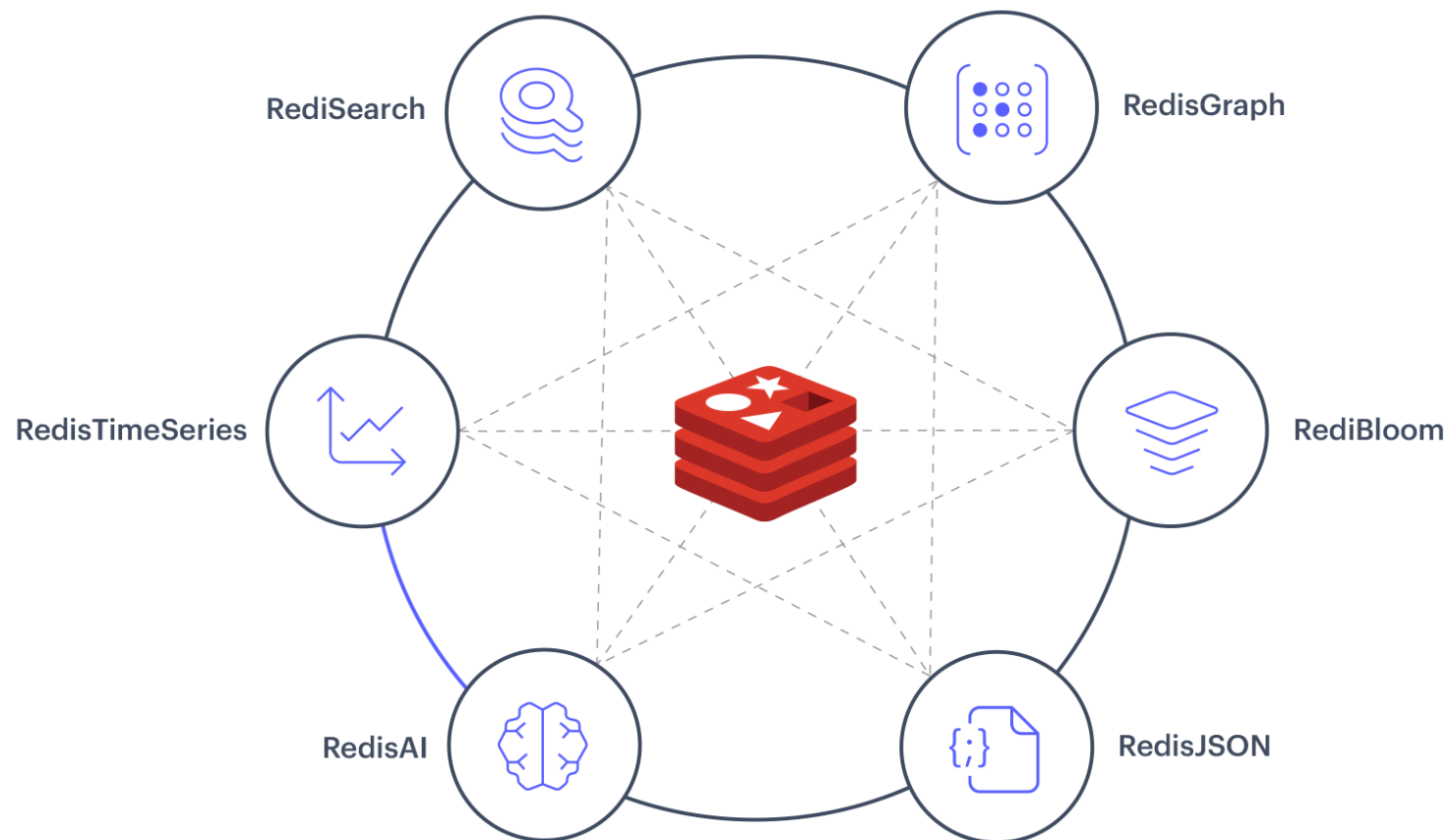
© 2023 Redis

Multiparadigma

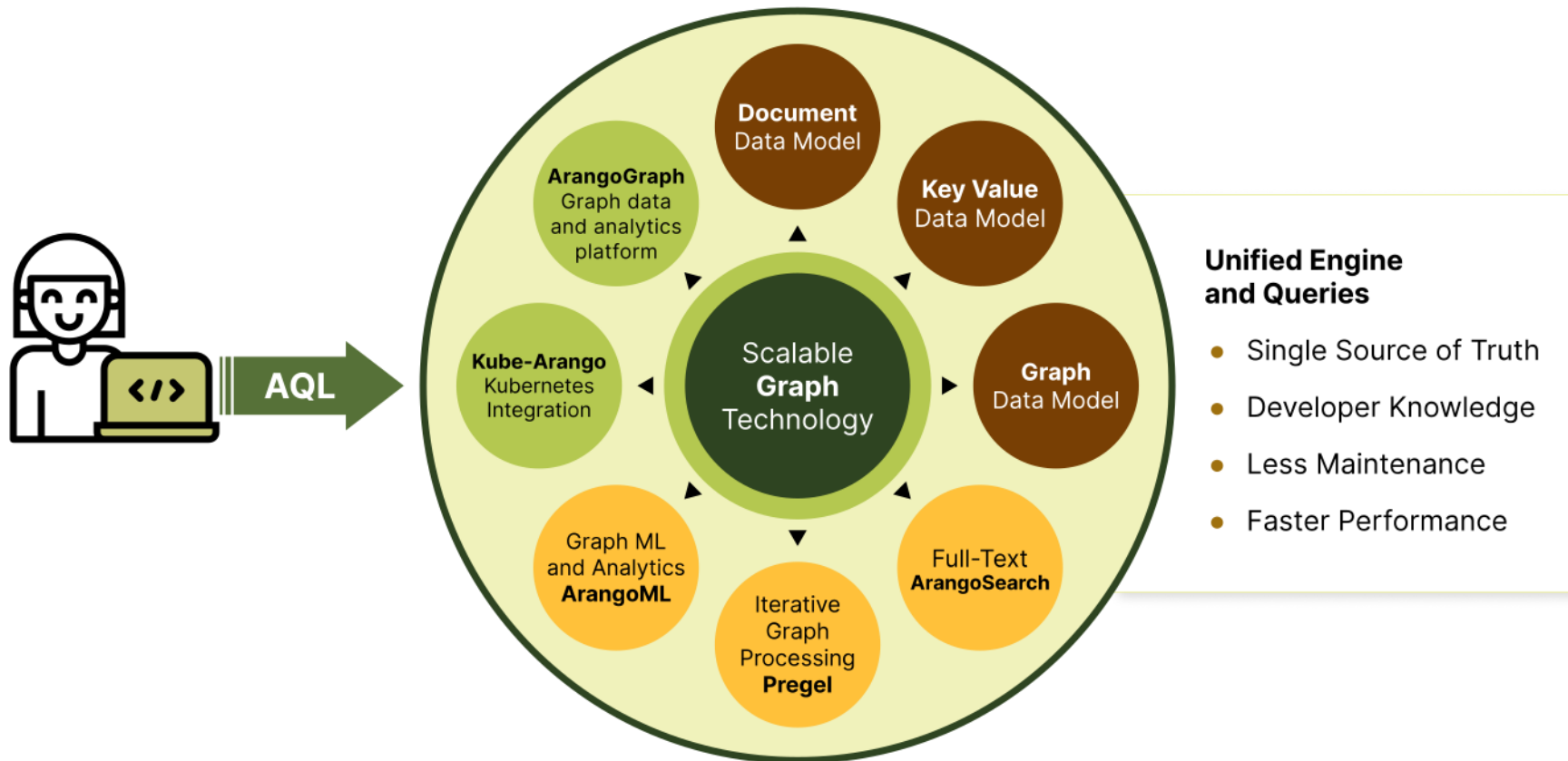
- Mezcla de **uno o más modelos en una misma plataforma**
 - Menos vendors
 - Más poder en una base de datos
 - Reducción de costos de networking
- Ejemplos
 - Arango: Graph+Key-value+Rel+Docs
 - MarkLogic: Docs+ACID+Text
 - Redis: Graph+Key+...



E.G. Redis

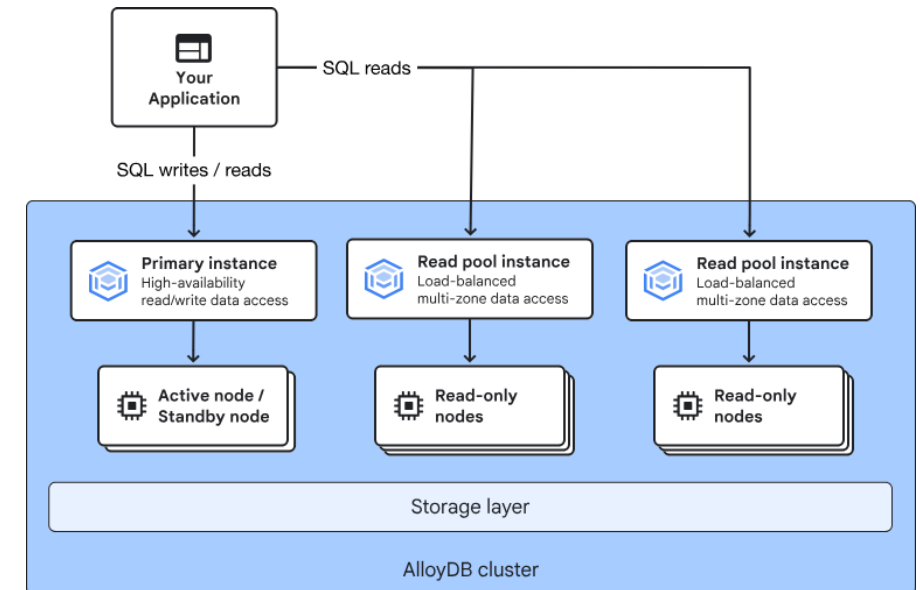


E.G. ArangoDB



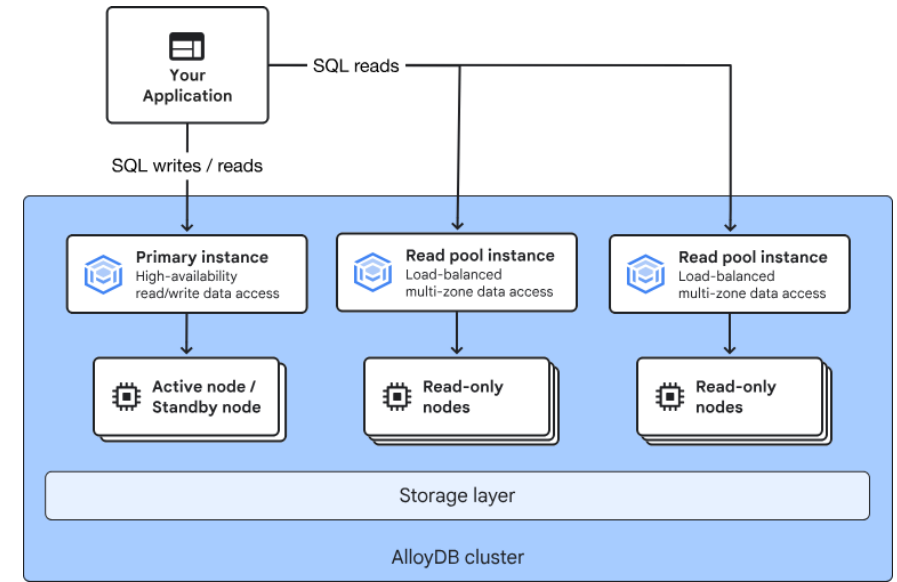
NewSQL

- Escalabilidad horizontal con propiedades SQL
 - Enfoque a escalabilidad global
 - Mantiene propiedades de consistencia
- Parte del “retorno a SQL”
- Mejora significativa en escalabilidad
 - Ojo que puede caer en ciertos casos
 - No está tan estandarizado
 - Puede no ser tan eficiente: van madurando



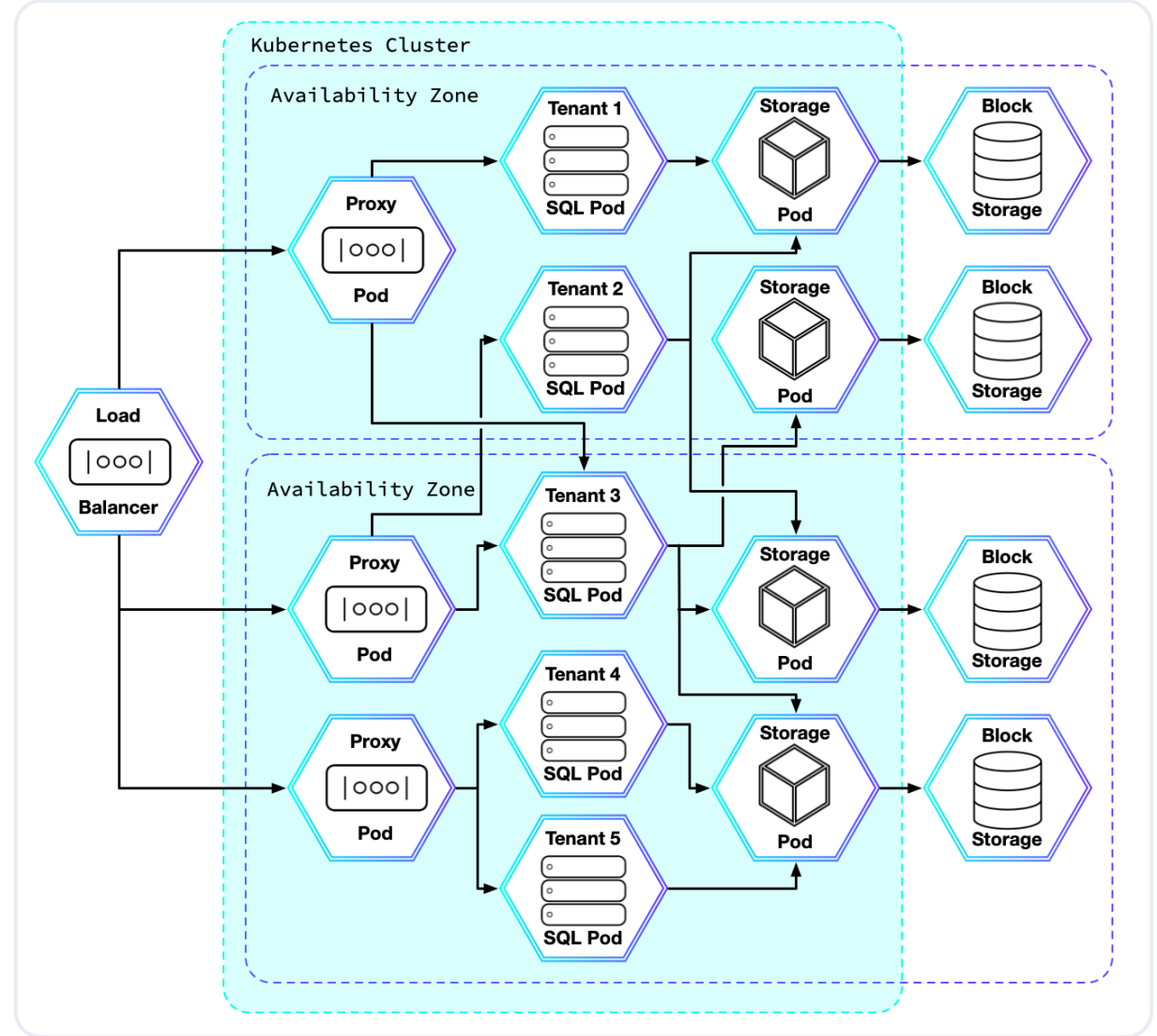
NewSQL

- Vendors
 - CockroachDB
 - Aurora
 - AlloyDB
 - GCP Spanner
 - Vitess (Con reservas en ACID)
 - VoltDB (No FK)
 - Alloydb



Ejemplo: CockroachDB

- Multizona
- Auto-healing
- Multicloud
- “Survivable”
 - Como el insecto



Consideraciones estratégicas

- Decidir en base a constraints
 - ¿Cómo se comporta mi dominio?
 - ¿Qué tipo de dato se requiere?
 - Reducir impedancia con sistema y propuesta
 - Escalabilidad
 - Tipos de consulta
 - Requerimiento evolutivo
 - Que sabe hacer mi equipo?
- Madurez del software
- Tiempo de implementación
- Requisitos de flexibilidad

Consideraciones tácticas

- Modelos de datos y queries:
 - ¿Filas? ¿Objetos? ¿Estructuras de datos? ¿Documentos? ¿Agregación?
- Durabilidad:
 - ¿El cambio en un valor debe persistirse inmediatamente? ¿En varias máquinas?
- Escalabilidad, Particiones, Consistencia:
 - ¿Un solo disco o varios?
 - ¿Read-heavy? ¿Write-heavy?
- ¿Cómo se coordinan los múltiples servidores?
- Transacciones:
 - ¿Qué propiedades ácidas son necesarias? ¿Puedo prescindir de algunas?
- ¿Carga basada en análisis de comportamiento de usuarios?